

UT_2.1_Arrays, Arreglos, Matrices

1 Introducción a las matrices

2 Trabajar con matrices

3 Añadir elementos a una matriz

4 Eliminar elementos de una matriz

5 Copiar algunos de los elementos de una matriz

6 Ordenar matrices

7 Usar matrices como una pila

8 Uso de matrices como cola

9 Búsqueda de elementos en matrices

10 Copiar elementos dentro de una matriz

11 Conversión de matrices en cadenas

1 Introducción a las matrices

Consideremos los usuarios de un sitio web de compras en línea. Por ejemplo, la edad de un usuario se representaría con un valor numérico, el nombre del usuario con una cadena y si el usuario se suscribió o no al boletín informativo mediante un valor booleano.

Pero para representar el carrito de la compra del usuario, por ejemplo, que en su forma más simple es una lista de nombres de artículos, no llegarás muy lejos solo con los tipos de datos primitivos, ya que entonces necesitarías definir una variable separada para cada entrada del carrito de la compra, como se muestra en Listado 1.

```
const shoppingCartItem1 = "Tocadiscos";      // primer artículo
const shoppingCartItem2 = "Altavoz";         // segundo artículo
const shoppingCartItem3 = "Preamplificador"; // tercer artículo
const shoppingCartItem4 = "Cables de altavoz"; // cuarto artículo
```

Listado 1 En algunos casos, los tipos de datos primitivos por sí solos no funcionan

Este proceso sería más que engorroso por varias razones: Por un lado, tendrías que proporcionar un número determinado de variables desde el principio, lo que a su vez significaría que un usuario no podría añadir más productos de los previstos al carrito de la compra. En segundo lugar, tareas como ordenar los productos por nombre también llevarían una cantidad de tiempo desproporcionada, ya que tendrías que comparar manualmente cada variable con todas las demás. En resumen, programar solo con los tipos de datos primitivos presentados hasta ahora sería extremadamente poco práctico.

Aquí es precisamente donde entran en juego las *matrices*. Las matrices no son más que listas; es decir, las matrices pueden contener no solo un valor, sino varios. En el ejemplo del carrito de la compra, se utilizaría una matriz para representar el carrito. Las *entradas* individuales de esta matriz serían entonces los nombres (y quizás los números de identificación) de los productos del carrito.

La forma más común y sencilla de crear una matriz en JavaScript es la *notación literal de matriz*. En este caso, basta con utilizar dos corchetes [] para definir el principio y el final de la matriz. Las *entradas* individuales (o *valores* o *elementos*) que se incluirán en la matriz se escriben entre estos dos corchetes, separadas por comas.

En el Listado 2 se muestra un ejemplo. En este caso, se crea una variable llamada `y` y se le asigna una matriz como valor. La matriz tiene cuatro entradas, a saber, las cadenas `,`, `,` y `,`.

```
const shoppingCart = [ // inicio de la definición de la matriz
  "Tocadiscos",        // primera entrada
  "Altavoz",           // segunda entrada
  "Preamplificador",   // tercera entrada
  "Cables de altavoz"   // cuarta entrada
];                      // fin de la definición de la matriz
```

Listado 2 Creación de una matriz mediante notación literal

A diferencia de otros lenguajes de programación, como Java, las matrices también pueden contener entradas de diferentes tipos. En otras palabras, una matriz puede contener números, cadenas o valores booleanos, objetos (más adelante se ofrece más información al respecto) e incluso otras matrices (véase Listado 3). Esto es posible en JavaScript debido a la tipificación flexible. Además, las matrices crecen dinámicamente, lo que significa que, al crear una matriz, no es necesario saber ni especificar cuántos elementos debe contener.

```
const values = [ // una matriz ...
  "John Doe",    // ... con una cadena ...
  22,            // ... un número ...
  true           // ... y un valor booleano
];
```

Listado 3 Ejemplo de definición de una matriz con valores numéricos

La estructura interna de las matrices

En ese tema aprenderás a insertar elementos en una matriz existente, eliminar elementos de una matriz, buscar elementos específicos dentro de una matriz, ordenar los elementos de una matriz según criterios específicos y mucho más. Sin embargo, hay algo que te mostraremos de antemano: la estructura interna de las matrices. No es particularmente complicada, pero es útil para comprender los ejemplos posteriores de este tema.

De hecho, las matrices tienen lo que se denomina una *estructura basada en índices*, lo que significa que cada elemento de una matriz se almacena en un índice concreto (también se podría decir que en una posición concreta). El recuento de estos índices comienza en 0, como es bastante habitual en informática: el primer elemento de la matriz se encuentra en el índice 0, el segundo elemento en el índice 1, y así sucesivamente hasta que el último elemento se encuentra en el índice $n-1$, donde n es la longitud de la matriz.

La estructura basada en índices es relevante durante la fase de desarrollo, ya que puede utilizar los índices para acceder específicamente a los elementos individuales de una matriz. Para ello, simplemente escriba el índice del elemento al que desea acceder entre corchetes después del nombre de la matriz correspondiente (véase Listado 4).

```
const item1 = shoppingCart[0]; // primera entrada
const item2 = shoppingCart[1]; // segunda entrada
const item3 = shoppingCart[2]; // tercera entrada
const item4 = shoppingCart[3]; // cuarta entrada
console.log(item1);            // «Tocadiscos»
console.log(item2);            // «Altavoz»
console.log(item3);            // «Preamplificador»
console.log(item4);            // «Cables de altavoz»
```

Listado 4 Acceso a los elementos de una matriz

Matrices multidimensionales

Ya has aprendido que puedes añadir números, cadenas y valores booleanos a una matriz. Sin embargo, también es posible añadir otras matrices a una matriz. Esto te permite crear *matrices multidimensionales* (es decir, matrices de matrices). Pueden ser útiles cuando se desea almacenar diferentes registros de los mismos datos en una matriz, de forma similar a una hoja de cálculo de Excel (este es un ejemplo de una *matriz bidimensional*).

En principio, también es posible utilizar matrices tridimensionales (es decir, matrices que contienen matrices que a su vez contienen matrices) o matrices con cualquier número de dimensiones. Sin embargo, a continuación solo hablaremos de las matrices bidimensionales, ya que son ideales para ilustrar el principio de las matrices multidimensionales.

Por ejemplo, consideremos el ejemplo del carrito de la compra, que se muestra de nuevo en el Listado 5. Se trata de una matriz unidimensional que solo contiene entradas de cadena que representan nombres de productos.

```
const shoppingCart = [  
  "Tocadiscos",  
  "Altavoz",  
  "Preamplificador",  
  "Cables de altavoz"  
];
```

Listado 5 Matriz unidimensional

Supongamos que ahora desea añadir propiedades adicionales a cada entrada para incluir no solo el nombre, sino también el precio y la cantidad del producto. Esto se podría modelar, por ejemplo, utilizando una matriz multidimensional, como se muestra en el Listado 6. Cada entrada de la matriz exterior es una matriz con tres elementos: el nombre del producto, el precio y la cantidad.

```
const shoppingCart = [  
  // inicio de la matriz externa  
  [  
    // primera entrada  
    "Tocadiscos", // primer elemento de la primera matriz  
    200,          // segundo elemento de la primera matriz  
    1,            // tercer elemento de la primera matriz  
  ],  
  // segunda entrada  
  [  
    "Altavoz", // primer elemento de la segunda matriz  
    400,       // segundo elemento de la segunda matriz  
    2,         // tercer elemento de la segunda matriz  
  ],  
  // tercera entrada  
  [  
    "Preamplificador", // primer elemento de la tercera matriz  
    80,                // segundo elemento de la tercera matriz  
    1,                 // tercer elemento de la tercera matriz  
  ],  
  // cuarta entrada  
  [  
    "Cables de altavoz", // primer elemento de la cuarta matriz  
    20,                  // segundo elemento de la cuarta matriz  
    2,                   // tercer elemento de la cuarta matriz  
  ],  
  // fin de la matriz externa  
];
```

Listado 6 Definición de una matriz multidimensional

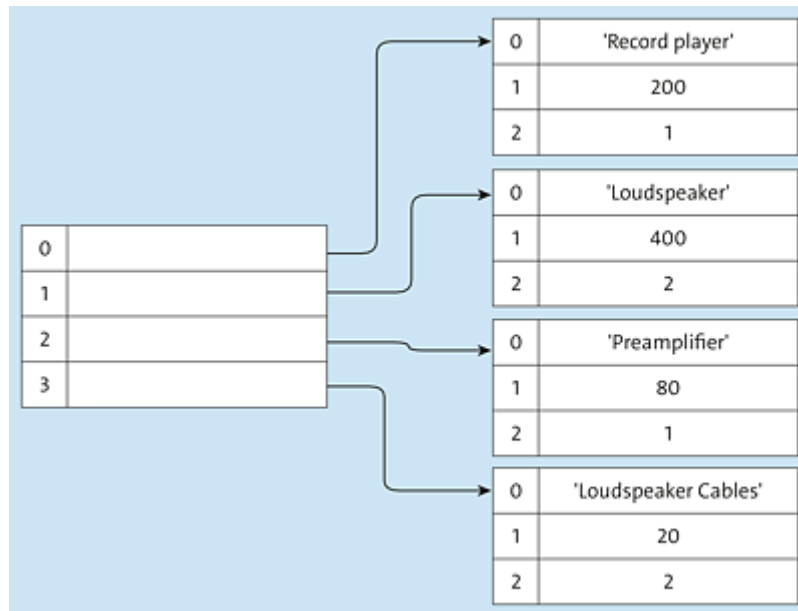


Figura 1 Estructura de las matrices multidimensionales

Al igual que con las matrices unidimensionales, se puede acceder a los valores individuales de una matriz multidimensional a través del índice. Sin embargo, dependiendo de la matriz y la dimensión a la que se desee acceder, será necesario proporcionar más detalles. Por ejemplo, para acceder al primer valor de la primera matriz (el valor), es necesario escribir `shoppingCart[0][0]`. El primero accede a la primera matriz (en la primera dimensión); el segundo accede a su primer valor (en la segunda dimensión). El listado 7 muestra cómo mostrar todos los valores de todas las matrices de la matriz de esta manera.

```
console.log(shoppingCart[0][0]); // Salida: Tocabiscos
console.log(shoppingCart[0][1]); // Salida: 200
console.log(shoppingCart[0][2]); // Salida: 1
console.log(shoppingCart[1][0]); // Salida: Altavoz
console.log(shoppingCart[1][1]); // Salida: 400
console.log(shoppingCart[1][2]); // Salida: 2
console.log(shoppingCart[2][0]); // Salida: Preamplificador
console.log(shoppingCart[2][1]); // Salida: 80
console.log(shoppingCart[2][2]); // Salida: 1
console.log(shoppingCart[3][0]); // Salida: Cables de altavoz
console.log(shoppingCart[3][1]); // Salida: 20
console.log(shoppingCart[3][2]); // Salida: 2
```

Listado 7 Acceso a los valores de una matriz multidimensional

2 Trabajar con matrices

En este punto te ofrecemos primero un breve resumen sobre cómo crear e inicializar matrices.

Creación e inicialización de matrices

En JavaScript, las matrices se pueden crear de dos maneras diferentes: mediante una función constructora y mediante la notación abreviada mostrada en el tema anterior, la notación literal de matriz. Utilizando la función constructora de matriz, se crea una matriz como se muestra en el Listado 8.

```
const names = new Array();
```

Listado 8 Creación de una matriz

Si conoce a priori el número de valores que se almacenarán en la matriz, también puede pasar este número a la función constructora como argumento, tal y como se muestra en el Listado 4.48 (aunque no es necesario hacerlo, ya que las matrices crecen dinámicamente).

```
const names = new Array(20);
```

Listado 9 Creación de una matriz con una longitud especificada

Si incluso conoce los valores que se almacenarán en la matriz, también puede pasarlos directamente como argumentos, como se muestra en el Listado 10.

```
const names = new Array("John", "James", "Peter");
```

Listado 10 Creación de una matriz con valores especificados

Llamada a la función constructora Array()

Ten en cuenta que si llamas a la función constructora `Array()` con un solo valor y este valor es un número, se utiliza para definir la longitud de la matriz. Si llamas a la función constructora con dos números como argumentos, la longitud de la matriz es implícitamente 2, y la matriz contiene los dos elementos numéricos.

Como ya sabe, la longitud de una matriz se puede determinar mediante la propiedad `length`.

```
const names = new Array("John", "James", "Peter");  
console.log(names.length); // Salida: 3
```

Listado 11 Determinación de la longitud de una matriz

Creación de matrices sin la palabra clave new

En el caso de las matrices, también se puede omitir `new` al llamar a la función constructora. Por lo tanto, el código fuente del Listado 4.50 podría reformularse de la siguiente manera:

```
const names = Array("John", "James", "Peter");
```

Dado que los desarrolladores suelen tener que trabajar con matrices en la práctica o necesitan utilizarlas con relativa frecuencia, JavaScript proporciona una notación breve muy práctica (notación literal de matriz) para crear matrices. Los valores individuales se escriben simplemente entre corchetes, como se muestra en el Listado 12.

```
const names = ["John", "James", "Peter"]; // Crear una matriz con valores específicos  
//  
const colors = []; // Crear una matriz vacía
```

Listado 12 Creación de una matriz utilizando notación abreviada

Convenientemente, la notación literal de matriz se puede combinar con la notación literal de objeto para, por ejemplo, crear una matriz de objetos, como en el Listado 13.

```
const contacts = [  
  {  
    firstName: "John",  
    lastName: "Doe",  
    email: "john.doe@javascripthandbuch.de"  
  },  
  {  
    firstName: "James",  
    lastName: "Doe",  
    email: "james.doe@javascripthandbuch.de"  
  },  
  {  
    firstName: "Peter",  
    lastName: "Doe",  
    email: "peter.doe@javascripthandbuch.de"  
  }  
];
```

Diferentes tipos en matrices

A diferencia de otros lenguajes, JavaScript ofrece la posibilidad de almacenar valores de diferentes tipos en una sola matriz. Esto significa, por ejemplo, que se pueden almacenar tanto cadenas como números en la misma matriz (véase el Listado 14).

```
const values = [];  
values[0] = "John";  
values[1] = 4711;  
values[2] = true;
```

Listado 14 Las matrices pueden contener valores de diferentes tipos de datos

Ahora que ya sabes cómo crear matrices, te mostraremos lo que puedes hacer con ellas. Las posibilidades incluyen lo siguiente:

Aspectos funcionales

Las matrices tienen varios métodos que admiten la programación funcional.

En la tabla 1 se presenta una descripción general de los métodos más importantes de las matrices.

Método	Descripción
concat()	Añade elementos o matrices a una matriz existente
filter()	Filtra elementos de la matriz basándose en un criterio de filtrado pasado en forma de función
forEach()	Aplica una función pasada a cada elemento de la matriz
join()	Convierte una matriz en una cadena
map()	Asigna los elementos de una matriz a nuevos elementos basándose en una función de conversión pasada
pop()	Elimina el último elemento de una matriz
push()	Inserta un nuevo elemento al final de la matriz
reduce()	Combina los elementos de una matriz en un solo valor basado en una función pasada
reverse()	Invierte el orden de los elementos de la matriz
shift()	Elimina el primer elemento de una matriz
slice()	Corta elementos individuales de una matriz
splice()	Añade nuevos elementos en cualquier posición de la matriz
sort()	Ordena la matriz

Tabla 1 Los métodos más importantes de las matrices

Se accede a los elementos individuales de la matriz a través de este índice, como se muestra en el listado 15. Para ello, se utilizan corchetes para especificar el índice al que se desea acceder. (El acceso a un índice inexistente devuelve `undefined` como valor de retorno).

```
const todoList = [
```

```
"Limpiar el baño", // elemento en el índice 0
"Ir de compras", // elemento en el índice 1
"Ordenar", // elemento en el índice 2
"Cortar el césped" // elemento en el índice 3
];
```

Listado 15

Para iterar sobre los elementos individuales de una matriz, puede utilizar un bucle for, como se muestra en el Listado 16.

```
for (let i = 0; i < todoList.length; i++) {
    console.log(todoList[i]);
}
```

Listado 16 Acceso a elementos individuales de una matriz mediante un bucle y un índice

3 Añadir elementos a una matriz

JavaScript proporciona varios métodos para añadir elementos a una matriz. Estos métodos permiten insertar elementos al principio o al final de una matriz, o en cualquier posición de la misma. A continuación, presentaremos los métodos adecuados.

Añadir un elemento al final de una matriz

El método más utilizado cuando se trabaja con matrices es probablemente el método `push()`. Se puede utilizar para añadir elementos al final de una matriz. Se puede pasar un elemento o varios elementos como argumentos. El método devuelve la nueva longitud de la matriz.

En el Listado 17 se muestra un ejemplo. Utiliza `push()` para añadir dos veces una cadena a la matriz `todoList` y, a continuación, añadir dos cadenas en una sola llamada. Después de cada llamada, la longitud de la matriz se asigna a la variable `length`, que toma los valores 1, 2 y 4 sucesivamente.

```
const todoList = [];
let length;
length = todoList.push("Limpiar el baño");
console.log(length); // 1
length = todoList.push("Ir de compras");
console.log(length); // 2
length = todoList.push("Ordenar", "Cortar el césped");
console.log(length); // 4
console.log(todoList); // [«Limpiar el baño», «Ir de compras»,
// «Ordenar», «Cortar el césped»]
```

Listado 17 Añadir elementos a una matriz mediante el método `push()`

Añadir un elemento al principio de una matriz

Si desea insertar nuevos elementos al principio de una matriz en lugar de al final, puede utilizar el método `unshift()`. Básicamente, este método funciona como el método `push()`, con la diferencia de que cambia las asignaciones de índice de todos los elementos siguientes. Al método `push()` también se le puede pasar un solo elemento o varios elementos separados por comas, que luego se añaden al principio de la matriz en orden. Una vez más, se devuelve la nueva longitud de la matriz. El Listado 18 muestra cómo se añaden dos nuevos elementos a la matriz `todoList` individualmente y luego otros dos elementos en un solo paso.

```
const todoList = [];
let length;
length = todoList.unshift("Limpiar el baño");
```

```

console.log(length); // 1
length = todoList.unshift("Ir de compras");
console.log(length); // 2
length = todoList.unshift("Ordenar", "Cortar el césped");
console.log(length); // 4
console.log(todoList); // [«Ordenar», «Cortar el césped»,
// «Ir de compras», «Limpiar el baño»]

```

Listado 18 Añadir elementos a una matriz utilizando el método `unshift()`

Insertar elementos en cualquier posición de la matriz

A veces, se necesita flexibilidad para añadir elementos no solo al principio o al final de una matriz, sino en posiciones arbitrarias. Esto se puede conseguir utilizando el método `splice()`.

Varios usos del método `splice()`

Como verás a lo largo de este tema, el método `splice()` se puede utilizar no solo para insertar elementos, sino también para eliminarlos o sustituirlos. Pero lo primero es lo primero: primero te mostraremos cómo utilizar `splice()` para añadir elementos en ubicaciones arbitrarias de la matriz.

Para ello, se pasan varios argumentos al método: El **primer** argumento representa la **posición** en la matriz desde la que se deben añadir los nuevos elementos. El **segundo** argumento representa el número de elementos que se deben eliminar de la matriz. Por ahora, siempre dejamos este argumento en el valor 0, ya que solo queremos utilizar `splice()` para añadir elementos (no para sustituir o eliminar elementos). Todos los demás argumentos representan los elementos que se deben añadir a la matriz.

El listado 19 muestra de nuevo un ejemplo de código fuente,

```

const todoList = [
  "Limpiar el baño",
  "Ir de compras",
  "Ordenar",
  "Cortar el césped"
];
todoList.splice(
  2, // Índice desde el que insertar elementos
  0, // Número de elementos que se van a eliminar
  "Pintar el garaje" // Elemento que se va a añadir
);
console.log(todoList);
// [
//   «Limpiar el baño»,
//   «Ir de compras»,
//   «Pintar el garaje»,
//   «Ordenar»,
//   «Cortar el césped»
// ]
todoList.splice(
  2, // Índice desde el que insertar elementos
  0, // Número de elementos que se van a eliminar
  "Colocar césped", // Elementos que se van a ...
  "Diseñar huerto" // ... añadir
);
console.log(todoList);
// [
//   «Limpiar el baño»,
//   «Ir de compras»,
//   «Colocar césped»,
//   «Hacer el huerto»,

```



```
// «Pintar el garaje»,  
// «Ordenar»,  
// «Cortar el césped»  
// ]
```

Listado 19 Añadir elementos a una matriz utilizando el método splice()

4 Eliminar elementos de una matriz

Al igual que para añadir elementos a una matriz, también hay diferentes formas de eliminar elementos de una matriz. Aquí también tienes varias opciones: ¿quieres eliminar un elemento del final o del principio de una matriz? ¿Quieres eliminar toda una parte del medio de la matriz? Te lo mostraremos en las siguientes secciones.

Eliminar el último elemento de una matriz

Al igual que el método push() se utiliza para añadir elementos, el método pop() se utiliza para eliminar elementos de una matriz. Es decir, este método le permite eliminar el último elemento de una matriz. Como valor de retorno, el método devuelve el elemento eliminado o, si no hay elementos en la matriz, el valor undefined. En el Listado 20 se muestra un ejemplo

```
const todoList = [  
  "Limpiar el baño",  
  "Ir de compras",  
  "Ordenar",  
  "Cortar el césped"  
];  
const item1 = todoList.pop();  
console.log(item1); // Salida: Cortar el césped  
const item2 = todoList.pop();  
console.log(item2); // Salida: Ordenar  
const item3 = todoList.pop();  
console.log(item3); // Salida: Ir de compras  
const item4 = todoList.pop();  
console.log(item4); // Salida: Limpiar el baño  
const item5 = todoList.pop();  
console.log(item5); // Salida: undefined
```

Listado 20 El método pop() elimina el último elemento de una matriz

Eliminar el primer elemento de una matriz

Si desea eliminar no el último elemento de una matriz, sino el primero, utilice el método shift() en lugar del método pop(). Básicamente, funciona de forma opuesta al método unshift(); es decir, elimina el primer elemento de una matriz y lo devuelve. Si no hay elementos en la matriz, el método, al igual que pop(), devuelve el valor undefined.

En el Listado 21 se muestra un ejemplo de esto. Aquí, los primeros elementos se eliminan secuencialmente de la matriz y se muestran.

```
const todoList = [  
  "Limpiar el baño",  
  "Ir de compras",  
  "Ordenar",  
  "Cortar el césped"  
];  
const item1 = todoList.shift();  
console.log(item1); // Salida: Limpiar el baño  
const item2 = todoList.shift();  
console.log(item2); // Salida: Ir de compras
```

```
const item3 = todoList.shift();
console.log(item3); // Salida: Ordenar
const item4 = todoList.shift();
console.log(item4); // Salida: Cortar el césped
const item5 = todoList.shift();
console.log(item5); // Salida: indefinido
```

Listado 21 El método `shift()` elimina el primer elemento de una matriz

Eliminar algunos de los elementos de una matriz

Cuando introdujimos el método `splice()` en el contexto de añadir elementos a una matriz, mencionamos que también se pueden eliminar elementos de una matriz mediante este método. Recuerde que si `splice()` se utiliza para añadir elementos, el segundo argumento se establece en el valor 0 y, a continuación, se le pasan los elementos que se van a añadir. Si, por el contrario, desea utilizar `splice()` para eliminar elementos, los parámetros se establecen de forma ligeramente diferente, como se muestra en el Listado 30.

El primer argumento representa la posición en la matriz desde la que se van a eliminar los elementos, y el segundo argumento representa el número de elementos que se van a eliminar. A diferencia de cuando se añaden elementos, no es necesario pasar más argumentos (porque no se van a añadir nuevos elementos).

En el Listado 22, los dos elementos centrales de la matriz `todoList` se eliminan de esta manera: `splice(1, 2)` elimina dos elementos a partir de la posición 1. Como valor de retorno, `splice()` devuelve una matriz que contiene los elementos eliminados

```
const todoList = [
  "Limpiar el baño",
  "Ir de compras",
  "Ordenar",
  "Cortar el césped"
];
const deletedTodos = todoList.splice(1, 2);
console.log(deletedTodos); // ["Ir de compras", "Ordenar"]
console.log(todoList); // ["Limpiar el baño", "Cortar el césped"]
```

Listado 22 El método `splice()` se puede utilizar para eliminar elementos de una matriz

5 Copiar algunos de los elementos de una matriz

Ya has visto cómo se pueden eliminar algunos de los elementos de una matriz utilizando el método `splice()`. Por otro lado, el método `slice()`, que presentaremos ahora, te permite copiar algunos de los elementos de una matriz. El valor devuelto es una matriz con copias de los elementos correspondientes. Los elementos originales permanecen en la matriz.

Se pueden pasar dos argumentos al método. El **primer** argumento indica la posición desde la que debe comenzar la extracción de los elementos, y el **segundo** argumento (opcional) indica la posición en la que debe terminar la extracción de los elementos. El listado 23 muestra algunos ejemplos.

```
const todoList = [
  "Limpiar el baño",
  "Ir de compras",
  "Ordenar",
  "Cortar el césped"
];
const sliced1 = todoList.slice(1); // comenzando por el segundo elemento
console.log(sliced1); // ["Ir de compras", "Ordenar", "Cortar el césped"]

const sliced2 = todoList.slice(2); // comenzando por el tercer elemento
```

```

console.log(sliced2); // [«Ordenar», «Cortar el césped»]

const sliced3 = todoList.slice(0, 2); // primer y segundo elemento
console.log(sliced3); // [«Limpiar el baño», «Ir de compras»]

const sliced4 = todoList.slice(2, 4); // tercer y cuarto elemento
console.log(sliced4); // [“Ordenar”, «Cortar el césped»]

// La matriz original permanece sin cambios:
console.log(todoList); // [«Limpiar el baño», «Ir de compras»,
                        // “Ordenar”, «Cortar el césped»]

```

Listado 23 El método slice() se puede utilizar para extraer partes de una matriz

Como se muestra, el método slice() se puede invocar con valores numéricos positivos como argumentos, pero también con números negativos, lo que a su vez afecta a la parte de la matriz que se extrae.

La tabla 2 muestra las diferentes combinaciones posibles y sus respectivos significados. En el listado 24 se muestran algunos ejemplos.

Primer parámetro	Segundo parámetro	Significado
Valor positivo	Valor positivo	Extrae la parte de la matriz desde el índice definido por el primer argumento hasta el índice definido por el segundo parámetro.
Valor positivo	No especificado	Extrae la parte de la matriz desde el índice definido por el primer argumento hasta el final de la matriz.
Valor negativo	No especificado	Extrae parte de la matriz a partir del final de la misma, donde el número de elementos extraídos viene determinado por el valor negativo. Por ejemplo, al llamar a slice(- 2) se extraen los dos últimos elementos de una matriz.
Valor positivo	Valor negativo	Extrae la parte de la matriz desde el índice definido por el primer argumento hasta la posición definida por el segundo argumento. Por ejemplo, al llamar a slice(1, -2) se extraen los elementos desde el índice 1 hasta el tercer índice desde el final.

Tabla 2 Las diferentes opciones de llamada del método slice()

```

const sliced5 = todoList.slice(-2); // los dos últimos elementos
const sliced6 = todoList.slice(1, // el segundo elemento desde el
// principio hasta ...
-1); // ... el segundo elemento desde el final
const sliced7 = todoList.slice(1, // el segundo elemento desde
// el principio hasta ...
-2); // ... el tercer elemento desde el final
const sliced8 = todoList.slice(1, // el segundo elemento desde el
// principio hasta ...
-3); // . . . el cuarto elemento desde el final
console.log(sliced5); // [«Ordenar», «Cortar el césped»]
console.log(sliced6); // [«Ir de compras», “Ordenar”]
console.log(sliced7); // [«Ir de compras»]
console.log(sliced8); // []

```

Listado 24 El comportamiento de slice() se puede controlar aún más especificando parámetros negativos

6 Ordenar matrices

Ordenar datos dentro de una matriz es relativamente sencillo. Hay dos métodos disponibles para este fin: `reverse()` y `sort()`. El primero simplemente invierte el orden de los elementos de la matriz; el segundo permite ordenar los elementos según criterios individuales específicos.

Invertir el orden de los elementos de una matriz

Si solo desea invertir el orden de los elementos de una matriz, la forma más sencilla es utilizar el método `reverse()`. Este método ordena los elementos en orden inverso (es decir, cambia la matriz) y, además, devuelve la matriz invertida como valor de retorno.

En el Listado 25 se muestra un ejemplo. La matriz `names` contiene inicialmente las cadenas John, James y Peter exactamente en ese orden. Después de llamar al método `reverse()`, las cadenas se enumeran en la matriz en orden inverso, es decir, Peter, James, John. Obviamente, no se puede hacer mucho con este método.

```
const names = ["John", "James", "Peter"];
names.reverse();
console.log(names); // Salida: Peter, James, John
```

Listado 25 Invertir el orden de los elementos de una matriz

Ordenar los elementos de una matriz según criterios específicos

El método `sort()` ofrece más flexibilidad porque permite definir opcionalmente un criterio de ordenación propio para influir en el orden. El criterio de ordenación se especifica en forma de función de comparación, que se pasa como argumento al método `sort()`. La función de comparación, a su vez, tiene dos parámetros y se invoca internamente por pares para los valores de la matriz cuando se llama a `sort()`. El valor de retorno de la función determina cuál de los dos valores es mayor que el otro:

El valor de retorno -1 significa que el primer valor es menor que el segundo.

El valor de retorno 1 significa que el primer valor es mayor que el segundo.

El valor de retorno 0 significa que ambos valores son iguales.

En el Listado 26 se muestra una función de comparación que compara, por ejemplo, valores numéricos según su tamaño.

```
function compare(valor1, valor2) {
  if (valor1 < valor2) {
    return -1; // El primer valor es menor que el segundo.
  } else if (valor1 > valor2) {
    return 1; // El primer valor es mayor que el segundo.
  } else {
    return 0; // Ambos valores son de igual tamaño.
  }
}
```

Listado 26 Estructura básica de una función de comparación

Esta función de comparación ahora se puede pasar al método `sort()`, como se muestra en el Listado 27. La matriz de valores inicialmente sin ordenar se ordena después de llamar a este método.

```
const values = [7, 6, 4, 8, 7, 2, 4];
values.sort(compare);
console.log(values); // 2, 4, 4, 6, 7, 7
```

Listado 27 Ordenación con tu propia función de comparación

Funciones como parámetros de otras funciones

Por cierto, lo que ocurre y se muestra en el Listado 27 es que, en JavaScript, puedes utilizar funciones como argumentos de otras funciones. Esta es una característica especial de JavaScript que no ofrece ningún otro lenguaje de programación. Volveremos a este tema más adelante. Por ahora, solo necesitas saber que las funciones se pueden pasar como argumentos a otra función. Sin embargo, ten en cuenta que aquí debes omitir los paréntesis. Si escribieras `values.sort(compare())`; en el ejemplo anterior, se llamaría a la función `compare()` y se pasaría como argumento el valor devuelto por la función, no la función en sí.

Ordenar objetos en matrices

En el Listado 28 se muestra un ejemplo un poco más complejo del uso del método `sort()`. Aquí puedes ver cómo ordenar una matriz que contiene objetos utilizando el método `sort()`. Para ello, hay tres funciones de comparación, `compareByFirstName()`, `compareByLastName()` y `compareByEmail()`, que comparan elementos individuales con respecto a diversas propiedades.

Dentro de las funciones de comparación, utilizamos el método `localeCompare()`, que está disponible para cadenas de forma predeterminada y tiene en cuenta la configuración específica de cada país para los formatos de fecha y hora, así como para ordenar cadenas. Este método también devuelve uno de los tres valores posibles: el valor `-1` para un caso en el que la cadena sobre la que se llama al método es «más pequeña» (en términos de ordenación alfabética) que la cadena pasada como argumento; el valor `1` para el caso en el que la cadena es «más grande»; y el valor `0` para el caso en el que ambas cadenas son de igual tamaño, es decir, en el que son la misma cadena.

```
const contacts = [
  {
    firstName: "John",
    lastName: "Doe",
    email: "john.doe@javascripthandbuch.de"
  },
  {
    firstName: "James",
    lastName: "Dean",
    email: "superjames@javascripthandbuch.de"
  },
  {
    firstName: "Peter",
    lastName: "Dickens",
    email: "dickens@javascripthandbuch.de"
  }
];

function compareByFirstName(contact1, contact2) {
  return contact1.firstName.localeCompare(contact2.firstName);
}

function compareByLastName(contact1, contact2) {
  return contact1.lastName.localeCompare(contact2.lastName);
}

function compareByEmail(contact1, contact2) {
  return contact1.email.localeCompare(contact2.email);
}

contacts.sort(compareByFirstName); // ordenar por nombre
console.log(contacts[0].firstName); // James
console.log(contacts[1].firstName); // John
console.log(contacts[2].firstName); // Peter

contacts.sort(compareByLastName); // ordenar por apellido
```

```

console.log(contacts[0].firstName); // James
console.log(contacts[1].firstName); // Peter
console.log(contacts[2].firstName); // John

contacts.sort(compareByEmail); // ordenar por dirección de correo electrónico
console.log(contacts[0].firstName); // Peter
console.log(contacts[1].firstName); // John
console.log(contacts[2].firstName); // James

```

Listado 28 Ordenar objetos dentro de una matriz

7 Usar matrices como una pila

En programación (y en informática en general), una pila se refiere a una estructura de datos que funciona según el principio «último en entrar, primero en salir» (LIFO), lo que significa que el último elemento añadido a la pila es el primero en eliminarse. Por lo general, el método para añadir elementos es `push()` y el método para eliminarlos es `pop()` (véase la figura 2).

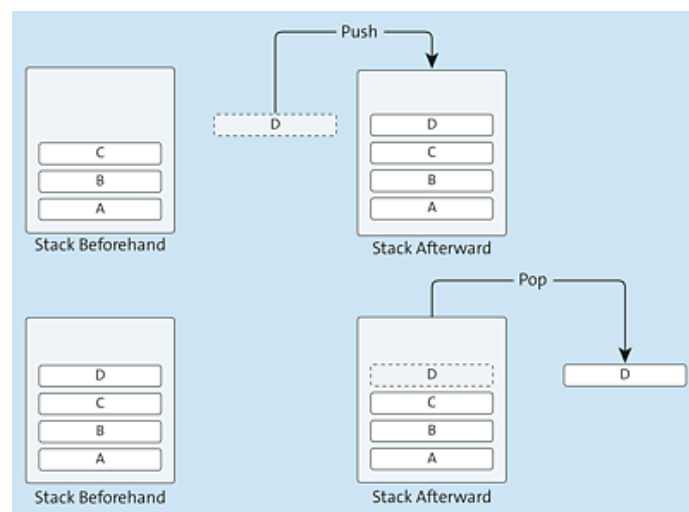


Figura 2 Cómo funcionan las pilas

Aunque JavaScript no proporciona ninguna estructura de datos de pila explícita en forma de objeto especial o similar, es posible emular el comportamiento descrito utilizando matrices. Para ello, se utilizan los métodos `push()` y `pop()`, que básicamente hacen lo mismo que las pilas. En el Listado 29 se muestra un ejemplo sencillo.

```

const stack = []; // Declaración de una matriz normal
stack.push(1); // Añadir un elemento ...
stack.push(2); // ... y otro más ...
stack.push(3); // ... y otro más ...
stack.push(4, 5, 6, 7, 8); // ... y varios de una sola vez
console.log(stack.pop()); // Devuelve el elemento que se
// añadió en último lugar: 8.

```

Listado 29 Ejemplo sencillo del uso de una matriz como pila

Ejemplo práctico de pilas

Un ejemplo práctico del uso de la estructura de datos pila (además de la pila de llamadas a funciones; véase el tema 3) es la implementación de una funcionalidad de deshacer, es decir, la posibilidad dentro de una aplicación de deshacer la última acción del usuario (véase la figura 3). Para ello, las acciones individuales se ordenan consecutivamente en la pila. Si el usuario desea ahora deshacer una acción, la acción correspondiente se toma de la pila y se ejecuta la parte correspondiente para deshacer la acción.

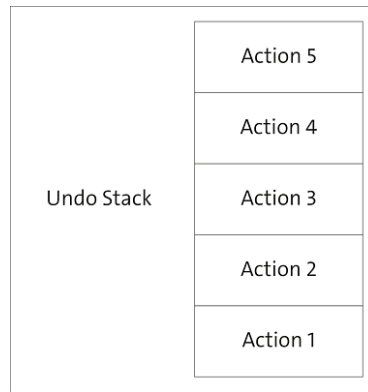


Figura 3 La funcionalidad de deshacer se implementa normalmente mediante una pila

Implementación alternativa de pilas

Como alternativa al uso de `push()` y `pop()`, se pueden utilizar los métodos `shift()` y `unshift()` para implementar una pila. La pila funciona entonces al revés: con `unshift()` se añaden elementos al principio y con `shift()` se elimina el primer elemento.

8 Uso de matrices como cola

Al igual que la pila, una cola también es una estructura de datos, pero funciona exactamente al revés: una cola siempre devuelve el elemento que se añadió como el primero de los elementos contenidos en la cola, mediante el principio primero en entrar, primero en salir (FIFO). En otras palabras, los elementos se añaden al final de una cola, pero se eliminan del principio de la cola (véase la figura 4). Por lo general, los métodos correspondientes se denominan `enqueue()` (para añadir elementos) y `dequeue()` (para eliminar elementos).

Las colas también se pueden emular utilizando matrices en JavaScript. Al igual que con las pilas, el método `push()` garantiza que los elementos se añadan al final de la matriz. El resto lo hace el método `shift()`, que devuelve y, al mismo tiempo, elimina el primer elemento de una matriz. El listado 30 muestra un ejemplo sencillo.

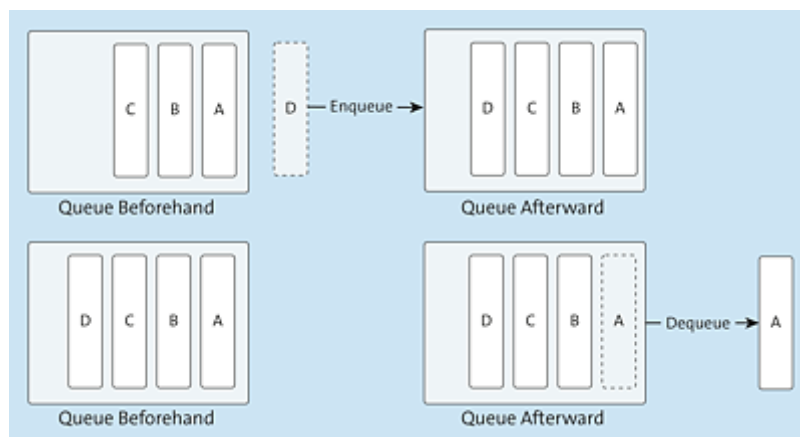


Figura 4 Cómo funcionan las colas

```
const queue = []; // Declaración de una matriz normal
queue.push(1); // Añadir un elemento ...
queue.push(2); // ... y otro ...
queue.push(3); // ... y otro más ...
queue.push(4, 5, 6, 7, 8); // ... y varios de una sola vez
console.log(queue.shift()); // Devuelve el elemento que se
// añadió primero: 1.
```

Listado 30 Ejemplo sencillo de uso de una matriz como cola

Implementación alternativa de colas

Como alternativa al uso de `push()` y `shift()`, por supuesto se puede utilizar `pop()` y `unshift()` para implementar una cola. La cola funciona entonces al revés: con `unshift()` se añaden elementos al principio y con `pop()` se elimina el último elemento.

9 Búsqueda de elementos en matrices

Los métodos `indexOf()`, `lastIndexOf()` y `find()` están disponibles para buscar elementos en matrices.

Búsqueda desde el principio de la matriz

Para el valor pasado, el método `indexOf()` determina la posición (es decir, el índice) en la que aparece por primera vez dentro de la matriz. Si no existe en la matriz, el método devuelve -1. Se puede utilizar un segundo parámetro opcional para controlar desde qué índice comenzar la búsqueda. En el Listado 31 se muestran algunos ejemplos.

```
const transactions = [
  -20.0, 500.50, -40.0, -34.50, 200, 500.50, -20, 200
];
console.log(transactions.indexOf(-20.0)); // Salida: 0
console.log(transactions.indexOf(500)); // Salida: -1
console.log(transactions.indexOf(200)); // Salida: 4
console.log(transactions.indexOf(200, 5)); // Salida: 7
```

Listado 31 El método `indexOf()` permite buscar elementos dentro de una matriz

Búsqueda desde el final de una matriz

El método `indexOf()` devuelve el primer índice en el que se ha encontrado el valor pasado en la matriz. No se tienen en cuenta las apariciones posteriores. Como alternativa, puede utilizar el método `lastIndexOf()`, que busca desde el final de una matriz y, en consecuencia, encuentra la última aparición del valor pasado en la matriz. Opcionalmente, al igual que con `indexOf()`, se puede utilizar un segundo parámetro para definir desde qué índice comenzar la búsqueda, y este método también devuelve -1 si el elemento buscado no aparece en la matriz (véase el Listado 32).

```
console.log(transactions.lastIndexOf(-20.0)); // Salida: 6
console.log(transactions.lastIndexOf(500)); // Salida: -1
console.log(transactions.lastIndexOf(200)); // Salida: 7
console.log(transactions.lastIndexOf(200, 5)); // Salida: 4
```

Listado 32 El método `lastIndexOf()` permite buscar elementos dentro de una matriz comenzando por el final

Otro método para matrices

Con la versión ES2016 (es decir, la sucesora de ES6 o ES2015), se introdujo el método `includes()` para matrices. Comprueba si el valor pasado está contenido en la matriz o no y devuelve un valor booleano correspondiente. Opcionalmente, se puede especificar el índice desde el que se va a buscar el elemento como segundo parámetro.

```
[4, 5, 6].includes(4); // true
[2, 7, 8].includes(4); // false
["John", "James"].includes("Peter"); // false
["John", "James"].includes("James"); // true
[4, 5, 6, 4711, 50, 87].includes(6, 2); // true
[4, 5, 6, 4711, 50, 87].includes(6, 4); // false
```

Listado 33 Comprobación sencilla de si una matriz contiene un elemento determinado

Búsqueda de elementos por criterio de búsqueda

Además de los métodos `indexOf()` y `lastIndexOf()`, también existen el método `findIndex()` y, desde ES2016, el método `find()`. Ambos métodos pasan una función como argumento, que se llama para cada elemento de la matriz y determina, mediante su valor de retorno (booleano), si el elemento respectivo cumple o no el criterio de búsqueda. El método `findIndex()` devuelve el índice de la primera aparición como resultado, mientras que el método `find()` devuelve el elemento en sí. En el Listado 43 se muestra un ejemplo. El criterio de búsqueda que se pasa aquí es una función que devuelve `true` para los números impares. El uso del método `find()` con esta función devuelve el valor 3 como argumento, ya que este es el primer valor impar de la matriz. El método `findIndex()` devuelve el valor 1, ya que 3 aparece por primera vez en el índice 1 de la matriz.

```
const numeros = [2,3,4,5,6,7,2,3,4,5]
//buscar el primer elemento impar
const result = numeros.find(element => element % 2 !== 0)
console.log(result); // 3

//buscar el índice del primer elemento impar
const result2 = numeros.findIndex(element => element % 2 !== 0);
console.log(result2); // 1
```

Listado 34 Los métodos `find()` y `findIndex()` utilizan un criterio de búsqueda para encontrar la primera ocurrencia que coincida con dicho criterio

10 Copiar elementos dentro de una matriz

Con la versión 6 del estándar ECMAScript, se han añadido algunos métodos nuevos para trabajar con matrices. Uno de ellos es el método `copyWithin()`, que se puede utilizar para copiar elementos dentro de una matriz.

Como argumentos, se le pasa el índice al que se van a copiar los elementos, el índice desde el que se van a copiar los elementos y, opcionalmente, el índice en el que debe terminar la copia de los elementos. Los elementos que se encuentran actualmente en el lugar al que se van a copiar los elementos serán sustituidos por estos elementos.

El listado 35 muestra un ejemplo.

```
const todoList = [
  "Limpiar el baño",
  "Ir de compras",
  "Ordenar",
  "Cortar el césped"
];
todoList.copyWithin(
  0, // posición inicial de destino a la que se copiarán los elementos
  2, // posición inicial de origen desde la que se copian los elementos
  4 // posición final de origen hasta la que se copian los elementos
);
console.log(todoList);
// ["Ordenar", «Cortar el césped», "Ordenar", «Cortar el césped»]
```

Listado 35 Con `copyWithin()`, los elementos se pueden copiar dentro de una matriz

11 Conversión de matrices en cadenas

Para convertir matrices en una cadena (y enviarla a la consola, por ejemplo), hay varias opciones. Por un lado, puedes utilizar los métodos `toString()` y `toLocaleString()`. Por otro lado, las matrices ofrecen el método `join()`, que se puede utilizar para unir los elementos individuales de una matriz en

una cadena. En este caso, es posible especificar una cadena como parámetro que se utilizará como separador. El uso de estos métodos se muestra en el Listado 36.

```
const nombres = ["María", "James", "Peter", 1234567.89, new Date];

const cadenaNombres = nombres.toString();
console.log(cadenaNombres); // María,James,Peter,1234567.89,Tue Sep 23 2025 19:12:14
GMT+0200 (hora de verano de Europa central)

console.log(nombres.toLocaleString('es-ES'));
// "1.234.567,89" (en España: punto como separador de miles, coma decimal)
// "fecha en formato español"

console.log(nombres.toLocaleString('en-US'));
// "1,234,567.89" (en EE.UU.: coma como separador de miles, punto decimal)
// "fecha en formato estadounidense"

const valorNombres = nombres.valueOf();
console.log(valorNombres);

const nombresUnidos = nombres.join("-");
console.log(nombresUnidos); // Salida: María-James-Peter.....
```

Listado 36 Convertir una matriz en una cadena